

Αυτόνομο όχημα πυρόσβεσης



2ο Γυμνάσιο Μοσχάτου

Σχολικό έτος 2025 - 26

Περιγραφή του αυτόνομου οχήματος πυρόσβεσης

Ένας από τους μεγαλύτερους κινδύνους για την ανθρώπινη ζωή και το περιβάλλον είναι η εκδήλωση πυρκαγιάς. Ο κίνδυνος αυξάνεται τους καλοκαιρινούς μήνες με τους παρατεταμένους καύσωνες, ιδιαίτερα σε δασικές περιοχές. Πέρα από τα αναγκαία μέτρα πρόληψης, ώστε να μειωθεί ο κίνδυνος, είναι σημαντική η έγκαιρη επέμβαση πριν εξαπλωθεί η πυρκαγιά. Ο σκοπός του οχήματος που σχεδιάζουμε είναι η έγκαιρη επέμβαση σε περίπτωση πυρκαγιάς. Το όχημα θα μπορεί να περιπολεί σε άλση και πάρκα με καθορισμένες διαδρομές.

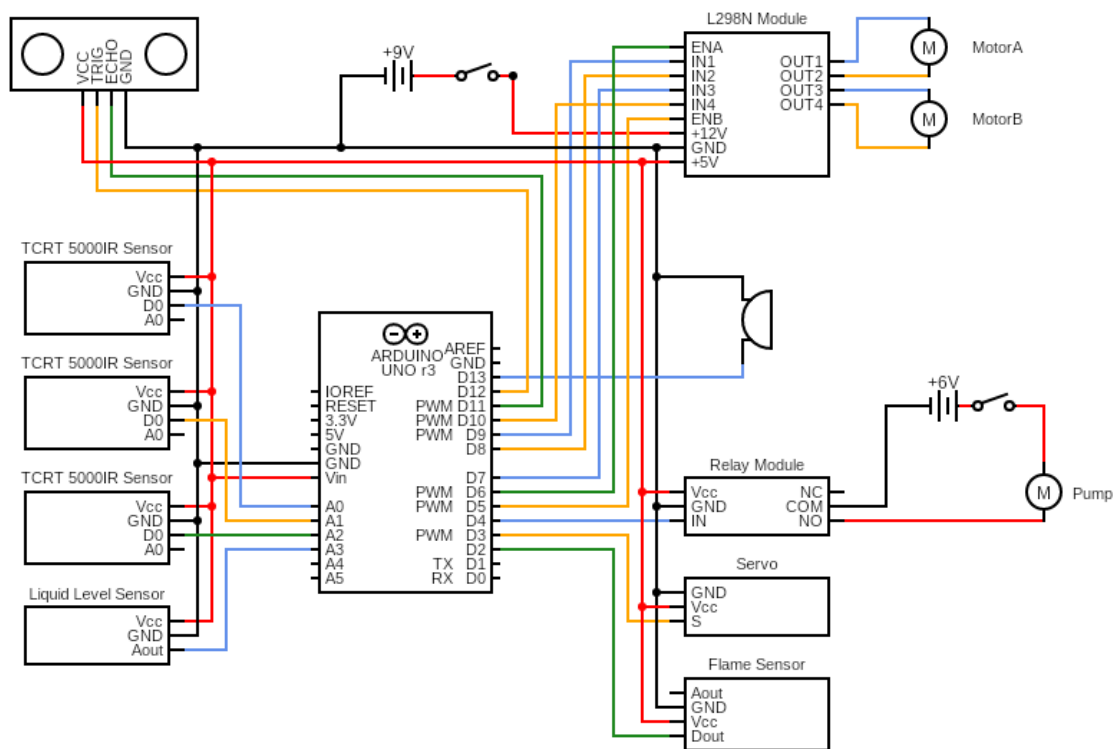
Το αυτόνομο πυροσβεστικό όχημα που σχεδιάσαμε έχει τη δυνατότητα να ακολουθεί μια μαύρη γραμμή στο δάπεδο του χώρου όπου θα κινηθεί. Η κίνηση διακόπτεται σε τακτά χρονικά διαστήματα για να σαρωθεί ο χώρος για την ύπαρξη φωτιάς. Στην περίπτωση που ανιχνευτεί φωτιά, το όχημα επεμβαίνει άμεσα με τη ρίψη νερού που υπάρχει στη δεξαμενή του. Αν το νερό τελειώσει, επιστρέφει στην αφετηρία της διαδρομής. Στο ίδιο σημείο σταματά κάθε φορά που ολοκληρώνει τη διαδρομή για να ελέγξει τη στάθμη του νερού της δεξαμενής.

Σχεδίαση του αυτόνομου οχήματος πυρόσβεσης

Για την υλοποίηση του οχήματος χρησιμοποιήθηκε μια πλατφόρμα με δύο κινητήρες DC, οι οποίοι ελέγχονται από μια πλακέτα με το ολοκληρωμένο ελέγχου L298N. Η ανίχνευση της διαδρομής του οχήματος πραγματοποιείται με τη βοήθεια τριών αισθητήρων υπερύθρων και η ανίχνευση τυχόν εμποδίων στη διαδρομή με ένα αισθητήρα υπερήχων. Η ανίχνευση για την ύπαρξη φωτιάς γίνεται από ένα αισθητήρα φλόγας, που περιστρέφεται με τη βοήθεια σερβό, για να σαρώσει το χώρο μπροστά από το όχημα. Στην βάση του αισθητήρα φλόγας στηρίζεται και το ακροφύσιο της αντλίας νερού, το οποίο περιστρέφεται ταυτόχρονα. Η ενεργοποίηση της αντλίας γίνεται μέσω ρελέ, μόλις ανιχνευθεί η ύπαρξη φωτιάς, οπότε διακόπτεται η περιστροφή του αισθητήρα και του ακροφυσίου και ξεκινά η πυρόσβεση. Η κατάσβεση σταματά όταν η φωτιά σβήσει ή όταν εξαντληθεί το νερό από τη δεξαμενή του οχήματος. Το όχημα συνεχίζει τη διαδρομή του μέχρι τη θέση εκκίνησης, όπου ελέγχει τη στάθμη του νερού στη δεξαμενή του, πριν επαναλάβει τον έλεγχο τη διαδρομής.

Το όχημα τροφοδοτείται από μια συστοιχία 6 μπαταριών AA, η οποίες περιέχονται σε μια μπαταριοθήκη με διακόπτη. Ξεχωριστή συστοιχία 4 μπαταριών AA τροφοδοτεί την αντλία νερού. Για τον έλεγχο του οχήματος χρησιμοποιείται η πλακέτα Arduino Uno.

Οι συνδέσεις των στοιχείων του οχήματος φαίνονται στο παρακάτω σχήμα:

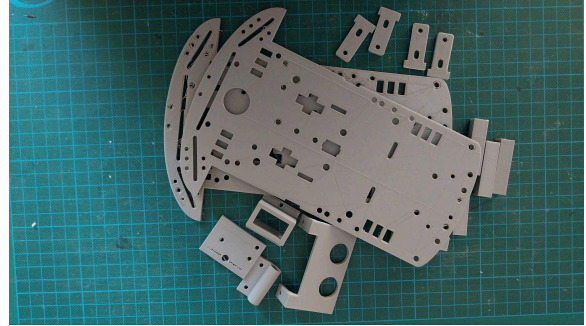


Line follower with fire fighter

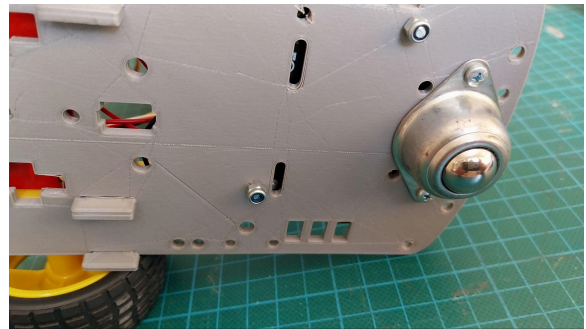
Το διάγραμμα των συνδέσεων δημιουργήθηκε με την δωρεάν online εφαρμογή δημιουργίας διαγραμμάτων ηλεκτρονικών κυκλωμάτων στη διεύθυνση www.circuit-diagram.org

Συναρμολόγηση του αυτόνομου οχήματος πυρόσβεσης

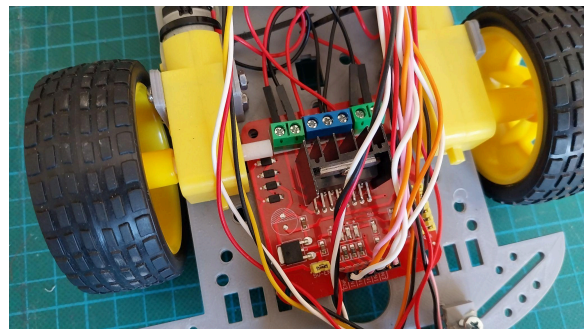
Η κατασκευή του οχήματος ξεκίνησε με την κατασκευή του σασί και των βάσεων για τη στήριξη των αισθητήρων και των υπόλοιπων στοιχείων του οχήματος με τη βοήθεια του 3D printer.



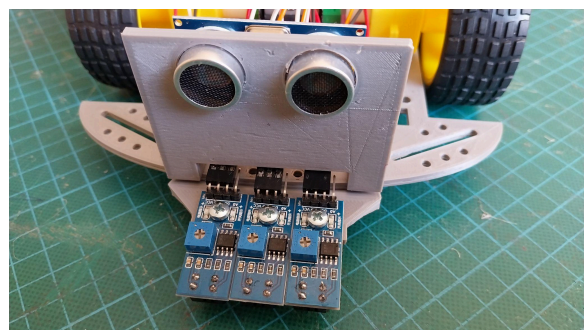
Το σασί του οχήματος αποτελείται από δύο όμοια μέρη, τα οποία τοποθετούνται το ένα πάνω από το άλλο και βιδώνονται μεταξύ τους με τους κατάλληλους αποστάτες. Στο κάτω τμήμα του σασί τοποθετήθηκαν οι κινητήρες DC με τους τροχούς τους και ο μεταλλικός σφαιρικός τροχός.



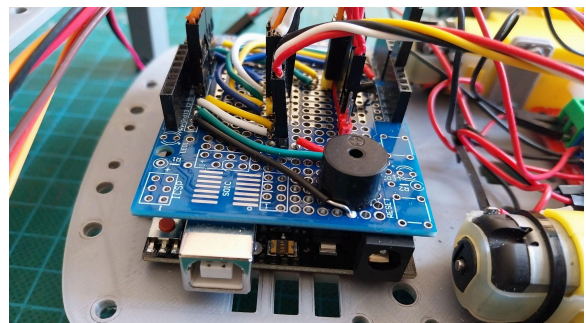
Ανάμεσα από τους κινητήρες τοποθετήθηκε η πλακέτα με το κύκλωμα οδήγησης των κινητήρων, όπου συνδέεται και η κύρια τροφοδοσία του οχήματος. Η πλακέτα αυτή τροφοδοτεί και την πλακέτα του Arduino καθώς και όλα τα υπόλοιπα στοιχεία του οχήματος.



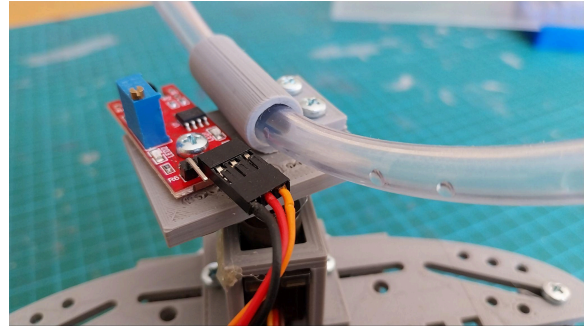
Στο μπροστινό μέρος τοποθετήθηκαν, με τη βοήθεια βάσεων που εκτυπώθηκαν από το 3D printer, οι αισθητήρες υπερύθρων για την ανάγνωση της γραμμής και ο αισθητήρας υπερήχων για την ανίχνευση εμποδίων.



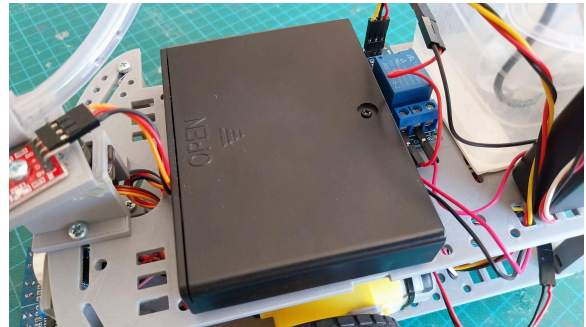
Στο πίσω μέρος σταθεροποιήθηκε η πλακέτα του Arduino και πάνω της τοποθετήθηκε η πλακέτα πρωτοτύπων, στην οποία σχηματίστηκαν όλες οι ηλεκτρικές συνδέσεις και κολλήθηκε και ο βομβητής του οχήματος.



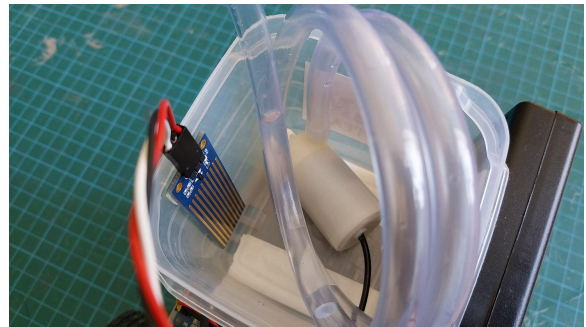
Στο πάνω τμήμα του σασί και στο μπροστινό μέρος τοποθετήθηκε το σερβό και στο περιστρεφόμενο τμήμα του η βάση με τον αισθητήρα φλόγας και το ακροφύσιο της αντλίας.



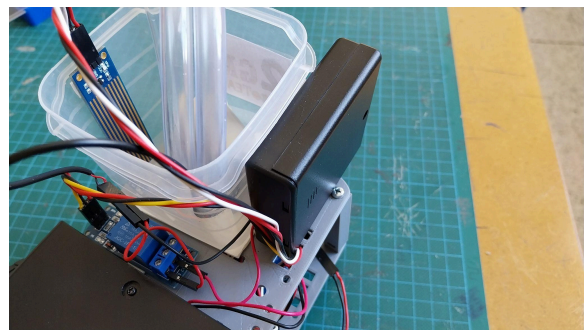
Πίσω τους τοποθετήθηκε η μπαταριοθήκη με τις 6 μπαταρίες AA που τροφοδοτούν το όχημα.



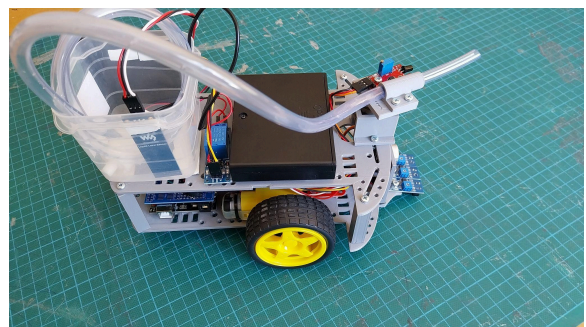
Στο πίσω μέρος κολλήθηκε η δεξαμενή νερού, που στο εσωτερικό της βρίσκονται η αντλία νερού και ο αισθητήρας στάθμης υγρών.



Δίπλα της τοποθετήθηκε η μονάδα ρελέ που ελέγχει την τροφοδοσία της αντλίας. Η τάση λειτουργίας της αντλίας παρέχεται από δεύτερη μπαταριοθήκη που περιέχει 4 μπαταρίες AA και κολλήθηκε στο πλάι της δεξαμενής.



Η ολοκλήρωση της συναρμολόγησης του αυτόνομου οχήματος πυρόσβεσης φαίνεται στη διπλανή εικόνα.



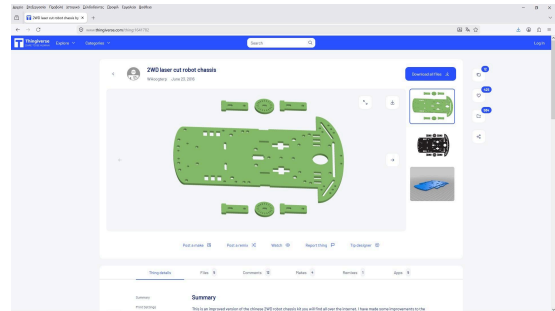
Τα υλικά που χρησιμοποιούμε στην κατασκευή, καθώς και το κόστος τους φαίνονται στον παρακάτω πίνακα:

Περιγραφή	Τεμάχια	Τιμή μονάδας	Κόστος
Arduino Uno (συμβατό)	1	12,80 €	12,80 €
Αισθητήρας υπερύθρων TCRT5000	3	1,60 €	4,80 €
Αισθητήρας υπερήχων HC-SR04	1	1,80 €	1,80 €
Πλακέτα οδήγησης κινητήρων με το L298	1	5,90 €	5,90 €
Κινητήρας DC με γρανάζια (125rpm)	2	1,80 €	3,60 €
Τροχός 66x26mm	2	1,20 €	2,40 €
Σφαιρικός μεταλλικός τροχός 15mm	1	1,60 €	1,60 €
Αισθητήρας φλόγας	1	3,50 €	3,50 €
Αισθητήρας στάθμης υγρών	1	3,50 €	3,50 €
Μονάδα ρελέ	1	1,90 €	1,90 €
Σερβό (MG90S Micro Servo)	1	4,90 €	4,90 €
Βομβητής 5V	1	0,60 €	0,60 €
Υποβρύχια αντλία	1	3,60 €	3,60 €
Πλακέτα πρωτοτύπου για Arduino	1	1,80 €	1,80 €
Arduino Long Headers 6 pin	2	0,25 €	0,50 €
Arduino Long Headers 8 pin	2	0,25 €	0,50 €
Μπαταριοθήκη 6 x AA με διακόπτη	1	1,80 €	1,80 €
Μπαταριοθήκη 4 x AA με διακόπτη	1	0,80 €	0,80 €
Μπαταρίες AA	10	0,40 €	4,00 €
Συνολικό κόστος			60,30 €

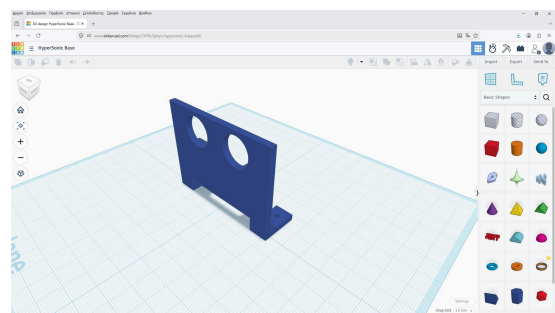
Στον παραπάνω πίνακα δεν περιλαμβάνεται το κόστος της καλωδίωσης και των απαραίτητων ακροδεκτων για τις συνδέσεις των στοιχείων του κυκλώματος, των βιδών και των παξιμαδιών τύπου M3 για τη συναρμολόγηση, καθώς και το κόστος της κατασκευής του σασί και των υπολοίπων εξαρτημάτων με τον 3D printer.

Κατασκευή των στοιχείων του οχήματος και της μακέτας για την επίδειξη της λειτουργίας

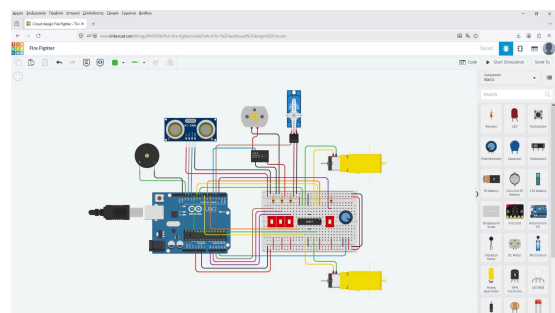
Για την κατασκευή του σασί του οχήματος με τη βοήθεια του 3D printer, χρησιμοποιήθηκε το έτοιμο σχέδιο **“2WD laser cut robot chassis”** από το αποθετήριο Thingiverse στη διεύθυνση: <https://www.thingiverse.com/thing:1641782>. Το εξάρτημα κατασκευάστηκε δύο φορές.



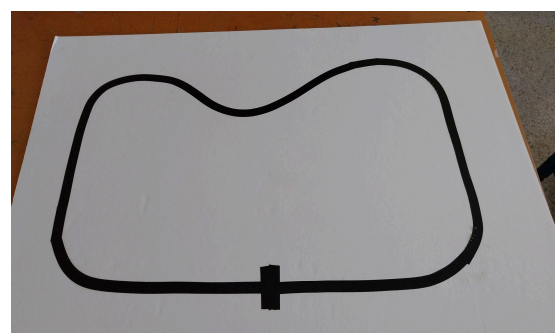
Με το ελεύθερο λογισμικό TinkerCAD σχεδιάστηκαν οι βάσεις των αισθητήρων υπερύθρων και του αισθητήρα υπερήχων, που τοποθετούνται στο μπροστά μέρος του οχήματος, η βάση στήριξης του αισθητήρα φλόγας και του ακροφυσίου της αντλίας, που περιστρέφεται με τη βοήθεια του σερβό, καθώς και οι αποστάτες, που τοποθετούνται μεταξύ των δύο στοιχείων του σασί. Στις κατάλληλες υποδοχές των αποστατών τοποθετήθηκαν παξιμάδια τύπου M3, για να βιδώσουν οι αντίστοιχες βίδες κατά τη συναρμολόγηση. Τα αρχεία τύπου *.stl των εξαρτημάτων βρίσκονται στο αποθετήριο του έργου, στη διεύθυνση: <https://github.com/2GymMoschatou/AutoFireFightingVehicle>



Το ελεύθερο λογισμικό TinkerCAD χρησιμοποιήθηκε και για την προσομοίωση της λειτουργίας και του προγραμματισμού των επιμέρους στοιχείων και του πλήρους κυκλώματος του οχήματος.



Για να ελέγξουμε τη λειτουργία του οχήματος κατασκευάσαμε μια διαδρομή πάνω σε χαρτί μακέτας με διαστάσεις 70x100 εκατοστά. Η διαδρομή σημειώθηκε με τη βοήθεια μαύρης μονωτικής ταινίας. Για να σημειώσουμε την αφετηρία, τοποθετήθηκε μια κάθετη γραμμή στη διαδρομή.



Προγραμματισμός του αυτόνομου οχήματος πυρόσβεσης

Στο πρώτο μέρος του προγράμματός εισάγουμε την κατάλληλη βιβλιοθήκη και δημιουργούμε το αντικείμενο του servo που χρειαζόμαστε.

```
//Including libraries
#include <Servo.h>
//Fire fighting settings
Servo myservo; //Creating servo
```

Στη συνέχεια ορίζουμε τους ακροδέκτες που θα χρησιμοποιεί το σύστημα πυρόσβεσης καθώς και τις απαραίτητες μεταβλητές για τον έλεγχο της λειτουργίας του με τον τύπο τους και τις αρχικές τιμές τους.

```
const int flamePin = 2; //flame sensor pin
const int relayPin = 4; //relay pin
const int beeperPin = 13; //beeper pin
const int waterPin = A3; //water level sensor pin
// variables
int flameState = 0; //flame status
int pos = 0; //servo initial position
int waterLevel = 0; //water level value
const int waterMin = 150; //minimum water tank level
const int waterMax = 300; //maximum water tank level
const int scanInterval=1000; //time between scans
unsigned long lastScanTime=0; //last scan time
```

Έπειτα ορίζουμε τους ακροδέκτες όπου θα συνδεθούν οι κινητήρες, και οι αισθητήρες που ελέγχουν την κίνηση του οχήματος. Επίσης ορίζουμε το όνομα και τον τύπο των μεταβλητών που απαιτούνται για τον έλεγχο της κίνησης.

```
//Line follower settings
//Motor A (right)
const int enA = 6;
const int motorPin1 = 9;
const int motorPin2 = 8;
//Motor B (left)
const int enB = 5;
const int motorPin3 = 7;
const int motorPin4 = 10;
//Ultrasonic Sensor
const int echoPin = 11;
const int trigPin = 12;
long duration, distance, cm;
//IRsensors
const int leftIR = A2;
const int middleIR = A1;
```



```
const int rightIR = A0;  
int leftID, middleID, rightID;
```

Στην ενότητα setup καθορίζουμε τον τρόπο λειτουργίας των ψηφιακών ακροδεκτών που έχουμε ορίσει ότι θα χρησιμοποιήσουμε. Επίσης συνδέουμε το servo με τον αντίστοιχο ακροδέκτη και καθορίζουμε την ταχύτητα λειτουργίας των κινητήρων με τις εντολές analogWrite.

```
void setup() {  
  //Set pins for Fire fighting  
  pinMode(flamePin, INPUT);  
  pinMode(relayPin, OUTPUT);  
  pinMode(beeperPin, OUTPUT);  
  myservo.attach(3);  
  //Set pins for motors  
  pinMode(motorPin1, OUTPUT);  
  pinMode(motorPin2, OUTPUT);  
  pinMode(motorPin3, OUTPUT);  
  pinMode(motorPin4, OUTPUT);  
  pinMode(enA, OUTPUT);  
  pinMode(enB, OUTPUT);  
  analogWrite(enA,200);  
  analogWrite(enB,200);  
  //Set pins for ultrasonic sensor  
  pinMode(echoPin, INPUT);  
  pinMode(trigPin, OUTPUT);  
  //Set pins as inputs for IR sensors  
  pinMode(leftIR, INPUT);  
  pinMode(middleIR, INPUT);  
  pinMode(rightIR, INPUT);  
}
```

Στην ενότητα loop διαβάζουμε πρώτα τη στάθμη του νερού στη δεξαμενή και την απόσταση του οχήματος από το πλησιέστερο εμπόδιο. Το πρόγραμμα ελέγχει την απόσταση του πλησιέστερου εμποδίου και αν είναι μικρότερη από όσο έχουμε ορίσει το όχημα σταματά, αλλιώς καλεί την υπορουτίνα για την κίνηση του οχήματος.

```
void loop() {  
  waterLevel = analogRead(waterPin);  
  cm = getDistance();  
  if (cm <= 15){  
    stop();  
  }  
  else {  
    move();  
  }  
  if (millis()-lastScanTime>=scanInterval){  
    stop();  
  }  
}
```

```

    fireDetect();
    lastScanTime=millis();
}
}

```

Όταν ο χρόνος κίνησης ξεπεράσει την τιμή που έχουμε ορίσει, το όχημα σταματά και ξεκινά η λειτουργία ανίχνευσης φλόγας. Αν ο αισθητήρας διαβάσει την ύπαρξη φλόγας, η ανίχνευση σταματά και εκτελείται η υπορουτίνα για την κατάσβεση, όσο υπάρχει ενεργή φωτιά και επαρκής ποσότητα νερού. Αλλιώς συνεχίζεται η κίνηση του οχήματος.

```

void fireDetect(){
  for (pos = 0; pos <= 180; pos += 15) {
    myservo.write(pos);
    delay(500);
    // read the flame state:
    flameState = digitalRead(flamePin);

    while (flameState == 1 && waterLevel > waterMin) {
      fireFight();
    }
  }
}

```

Στην υπορουτίνα κατάσβεσης, ενεργοποιείται η αντλία νερού μέσω του ρελέ και παράλληλα ακούγεται ο ήχος του συναγερμού για ένα δευτερόλεπτο. Η αντλία απενεργοποιείται και οι τιμές των αισθητήρων επανελέγχονται, ώστε να συνεχιστεί η διαδικασία ή να διακοπεί.

```

void fireFight(){
  // turn pump & beeper on:
  digitalWrite(relayPin, HIGH);
  tone(beeperPin, 1000, 1000);
  delay(1000);
  digitalWrite(relayPin, LOW);
  flameState = digitalRead(flamePin);
  waterLevel = analogRead(waterPin);
}

```

Στην υπορουτίνα ελέγχου της κίνησης του οχήματος, διαβάζουμε τις τιμές των αισθητήρων υπερέθρων, που παρακολουθούν τη γραμμή στο δάπεδο. Αν οι αισθητήρες υπερέθρων ανιχνεύσουν ότι η γραμμή βρίσκεται στο κέντρο του οχήματος, καλείται η υπορουτίνα για την κίνηση του οχήματος μπροστά, ενώ αν ανιχνεύσουν καμπύλη της διαδρομής καλούνται οι αντίστοιχες υπορουτίνες για τη στροφή του οχήματος. Στην περίπτωση που το όχημα βρεθεί εκτός γραμμής εκτελείται η υπορουτίνα για την κίνηση προς τα πίσω, μέχρι να ανιχνευτεί πάλι κάποια από τις υπόλοιπες καταστάσεις.

```

void move(){
  leftID = digitalRead(leftIR);
  middleID = digitalRead(middleIR);
}

```

```

rightID = digitalRead(rightIR);

if (leftID==HIGH && middleID==HIGH && rightID==HIGH){
  stop();
  checkWater();
}
else if (leftID==LOW && middleID==HIGH && rightID==LOW){
  forward();
}
else if (leftID==HIGH && middleID==HIGH && rightID==LOW){
  turnLeft();
}
else if (leftID==LOW && middleID==HIGH && rightID==HIGH){
  turnRight();
}
else if (leftID==HIGH && middleID==LOW && rightID==LOW){
  sharpTurnLeft();
}
else if (leftID==LOW && middleID==LOW && rightID==HIGH){
  sharpTurnRight();
}
else {
  backward();
}
}

```

Αν οι αισθητήρες υπερύθρων ανιχνεύσουν τη θέση εκκίνησης, η οποία σημειώνεται με μια κάθετη γραμμή στη διαδρομή, το όχημα σταματά και ελέγχει την ύπαρξη νερού στη δεξαμενή. Αν το νερό στη δεξαμενή είναι κάτω από το όριο που έχουμε θέσει ακούγεται ένας σύντομος ήχος για υπενθύμιση και το όχημα παραμένει ακίνητο. Όταν η στάθμη του νερού είναι μεγαλύτερη από το άνω όριο, το όχημα ξεκινά για να ξεπεράσει την κάθετη γραμμή στη διαδρομή και να συνεχίσει την ανίχνευση.

```

void checkWater(){
  waterLevel = analogRead(waterPin);
  if (waterLevel < waterMax){
    tone(beeperPin, 500, 300);
    stop();
  }
  if (waterLevel >= waterMax){
    go();
  }
}

```

Για κάθε κίνηση του οχήματος έχει οριστεί και μια διαφορετική υπορουτίνα. Οι υπορουτίνες αυτές καλούνται από την υπορουτίνα `move()` που ελέγχει την κίνηση ανάλογα με τις ενδείξεις των αισθητήρων υπερύθρων.

```

void forward(){
    // This code will move platform forward.
    digitalWrite(motorPin1, HIGH);
    digitalWrite(motorPin2, LOW);
    digitalWrite(motorPin3, HIGH);
    digitalWrite(motorPin4, LOW);
    delay(10);
}

void backward(){
    // This code will move platform backward.
    digitalWrite(motorPin1, LOW);
    digitalWrite(motorPin2, HIGH);
    digitalWrite(motorPin3, LOW);
    digitalWrite(motorPin4, HIGH);
    delay(5);
}

void turnLeft(){
    //This code will turn platform left.
    digitalWrite(motorPin1, HIGH);
    digitalWrite(motorPin2, LOW);
    digitalWrite(motorPin3, LOW);
    digitalWrite(motorPin4, LOW);
    delay(10);
}

void turnRight(){
    //This code will turn platform right.
    digitalWrite(motorPin1, LOW);
    digitalWrite(motorPin2, LOW);
    digitalWrite(motorPin3, HIGH);
    digitalWrite(motorPin4, LOW);
    delay(10);
}

void sharpTurnLeft(){
    //This code will sharp turn platform left.
    digitalWrite(motorPin1, HIGH);
    digitalWrite(motorPin2, LOW);
    digitalWrite(motorPin3, LOW);
    digitalWrite(motorPin4, HIGH);
    delay(10);
}

void sharpTurnRight(){
    //This code will sharp turn platform right.
    digitalWrite(motorPin1, LOW);

```

```

digitalWrite(motorPin2, HIGH);
digitalWrite(motorPin3, HIGH);
digitalWrite(motorPin4, LOW);
delay(10);
}

```

```

void stop(){
  //This code will stop platform.
  digitalWrite(motorPin1, LOW);
  digitalWrite(motorPin2, LOW);
  digitalWrite(motorPin3, LOW);
  digitalWrite(motorPin4, LOW);
  delay(2000);
}

```

Επίσης, υπορουτίνες έχουν οριστεί για την εκκίνηση του οχήματος από την αφετηρία, ώστε να ξεπεράσει την κάθετη γραμμή στη διαδρομή και για τη μέτρηση της απόστασης από κάποιο εμπόδιο.

```

void go(){
  digitalWrite(motorPin1, HIGH);
  digitalWrite(motorPin2, LOW);
  digitalWrite(motorPin3, HIGH);
  digitalWrite(motorPin4, LOW);
  delay(200);
}

```

```

long getDistance(){
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH);
  distance = (duration/2) / 29.1;
  return distance;
}

```


Βιβλιογραφία

1. [Βασίλης Νούσης, “ARDUINO για αρχάριους”, 2019](#)
2. [Εμμανουήλ Πουλάκης, “Προγραμματίζοντας με τον μικροελεγκτή Arduino”, 2015](#)
3. [Γεώργιος Γίδας, Ευάγγελος Μαντζάνας, “Εφαρμογές Arduino στο σχολικό εργαστήριο για το περιβάλλον”, 2021](#)
4. [Νικόλαος Φανουράκης, “Εκπαιδευτική Ρομποτική με τον μικροελεγκτή Arduino”, 2019](#)

Ιστοσελίδες

1. <https://www.thingiverse.com/thing:1641782>